

TRADUCTOR EDUCATIVO

Informe del proyecto

Python -> ASM x86_64 para Linux

Integrantes

Melo Reséndiz Jorge

Guerrero Serrano Jafeth Oswaldo

Paniagua Rico Juan Julián

TABLA DE CONTENIDO

1 Introducción	3
1.1 Antecedentes del Proyecto	3
2 Objetivos Generales	5
3 Gramatica	6
3.1 Tokens de la gramatica	6
3.2 Limitaciones actuales relevantes	6
4 Arquitectura del Compilador	7
5 Pruebas de casos de uso	8
5.1 Caso de uso: compilacion y ejecucion valida	8
5.2 Caso de uso: deteccion de errores sintacticos	8
5.3 Caso de uso: deteccion de errores semanticos	8
5.4 Caso de uso: inspeccion de salida ensamblador	8
6 Conclusiones	9
6.1 Conclusiones generales del proyecto	9
6.2 Logros principales	9
6.3 Estado del proyecto	9
6.4 Limitaciones actuales	10
6.5 Mejoras futuras recomendadas	10
6.6 Conclusion final	10

1 INTRODUCCIÓN

python2asm es un compilador educativo desarrollado en Java que traduce un subconjunto del lenguaje Python a código ensamblador x86_64 para Linux. Fue construido como proyecto académico para el curso de Programación de Sistemas Base 1 en la Universidad Autónoma de Tamaulipas, con el objetivo de demostrar en la práctica los conceptos fundamentales de la construcción de compiladores. El proyecto implementa un pipeline de compilación completo: desde la lectura del código fuente Python hasta la emisión de instrucciones ensamblador ejecutables. Para ello, utiliza ANTLR 4 como generador de parsers, construye un Árbol de Sintaxis Abstracta (AST) mediante el patrón Visitor, aplica análisis semántico básico, y finalmente genera código x86_64 compatible con NASM siguiendo las convenciones del ABI de Linux.

El subconjunto de Python soportado incluye variables, expresiones aritméticas, lógicas y de comparación, estructuras de control (if/elif/else, while, for con range()), tipos enteros, strings y booleanos, y salida básica con print(). Funcionalidades como funciones definidas por el usuario, listas y la forma completa de range(start, stop, step) quedan como trabajo pendiente. Fue desarrollado por Jorge Melo Reséndiz, Jafeth Oswaldo Guerrero Serrano y Juan Julián Paniagua Rico, bajo la supervisión del profesor Dante Adolfo Muñoz Quintero, iniciado en el octavo semestre periodo 2025-2 y finalizado oficialmente en noveno semestre de 2026.

El flujo sigue el esquema clásico de compiladores: preprocesamiento de indentación, análisis léxico y sintáctico con ANTLR 4.13.2, construcción de un AST y generación de código ASM. El objetivo principal es demostrar conceptos de compiladores (lexer, parser, AST, visitor y codegen) con un alcance controlado.

1.1 Antecedentes del Proyecto

Contexto académico

El proyecto surge en el marco de la materia Programación de Sistemas Base 1 (visible en el nombre del repositorio). Su propósito es aplicar de forma práctica los conceptos teóricos de construcción de compiladores: lexer, parser, AST, patrón Visitor y generación de código. README.md:41

Elección de Python como lenguaje fuente

Python presenta uno de los retos más particulares para el análisis sintáctico: la indentación significativa como delimitador de bloques, en lugar de llaves o palabras clave. Esto lo convierte en un caso de estudio interesante para implementar un preprocesador de tokens INDENT/DEDENT. architecture.md:5-9

Elección de x86_64 como arquitectura objetivo

x86_64 es la arquitectura dominante en sistemas Linux de escritorio y servidor. Compilar hacia ella implica trabajar directamente con el ABI System V (convenciones de registros, syscalls del kernel), lo que conecta la teoría de compiladores con la realidad del hardware. README.md:53-58

Herramientas previas que sustentan el proyecto

- ANTLR 4 (Terence Parr): generador de parsers ampliamente usado en proyectos reales y enseñanza de compiladores. README.md:93
- "Engineering a Compiler" (Cooper y Torczon): referencia académica estándar para el diseño de compiladores. README.md:99
- "Programming from the Ground Up" (Jonathan Bartlett): referencia para la generación de código ensamblador x86. README.md:101

Alcance controlado como decisión de diseño

Dado el contexto educativo, se optó por un subconjunto del lenguaje Python (sin funciones de usuario, sin listas, sin tipos flotantes) para mantener el proyecto manejable y enfocado en demostrar el pipeline completo, no en cubrir el 100% del lenguaje. informe.md:42-46

2

OBJETIVOS GENERALES

Objetivos Generales del Proyecto:

- Demostrar el pipeline completo de un compilador: implementar cada fase clasica (preprocesamiento -> analisis lexico/sintactico -> construccion de AST -> analisis semantico -> generacion de codigo), de forma modular y comprensible. README.md:41
- Traducir un subconjunto de Python a ensamblador x86_64 ejecutable en Linux: el compilador produce codigo NASM real que puede ensamblarse y ejecutarse directamente, no solo una representacion intermedia. README.md:50-51
- Resolver el problema de la indentacion significativa de Python: implementar un preprocesador que convierta la indentacion en tokens INDENT/DEDENT antes del analisis, uno de los retos mas particulares de Python frente a otros lenguajes. development.md:7-26
- Aplicar el patron de diseno Visitor sobre un AST tipado: separar la representacion del programa (AST) de las operaciones sobre el (analisis semantico, generacion de codigo), logrando un diseno extensible. README.md:119-121
- Soportar estructuras de control y tipos de datos basicos: cubrir if/elif/else, for, while, enteros, booleanos y strings como conjunto minimo funcional y representativo. README.md:45-49
- Proveer una base extensible para agregar nuevas características: la arquitectura esta disenada para que agregar construcciones (funciones, listas, etc.) siga un flujo sistematico de 6 pasos sin reescribir el sistema. development.md:92-116
- Servir como recurso educativo sobre construccion de compiladores: el proyecto esta orientado a estudiantes, con documentacion dirigida explicitamente a ese perfil y referencias academicas incluidas. README.md:32-37

3 GRAMATICA

3.1 Tokens de la gramatica

Categoría	Detalle
Palabras clave	for, while, if, elif, else, in, and, or, not, True, False.
Identificadores	IDENTIFIER con letras, numeros y guion bajo.
Literales	INT (enteros), STRING (comillas simples o dobles), booleanos.
Operadores	+, -, *, /, %, **, ==, !=, <, >, <=, >=.
Delimitadores	(,), ', ' ', ::
Indentación	INDENT y DEDENT insertados por el preprocesador.
Separación	NEWLINE, WS y comentarios # (ignorados por el lexer).

3.2 Limitaciones actuales relevantes

- range() se parsea con 1, 2 o 3 argumentos, pero la generación de código asume argumentos enteros literales.
- No hay definición de funciones de usuario ni estructuras complejas (listas, diccionarios, etc.).
- Sin tipos flotantes ni asignación compuesta.

Snippet 1 - range() con expresiones no literales:

```
inicio = 1
fin = 10
paso = 2
for i in range(inicio, fin, paso):
    print(i)
```

Snippet 2 - funciones de usuario y estructuras complejas no soportadas:

```
def suma(a, b):
    return a + b

nums = [1, 2, 3]
datos = {"x": 10}
print(suma(nums[0], datos["x"]))
```

Snippet 3 - flotantes y asignación compuesta:

```
x = 3.14
y = 2
x += y
print(x)
```

4 ARQUITECTURA DEL COMPILADOR

La arquitectura se organiza como un pipeline multi-fase:

- Preprocesamiento: `util.PythonIndentPreprocessor` convierte indentación en INDENT/DEDENT.
- Lexer/Parser (ANTLR4): `grammar/PythonSubset.g4` tokeniza y produce el parse tree.
- AST: `parser.ASTBuilder` transforma el parse tree en un AST con nodos como `ProgNode`, `AssignNode`, `IfNode`, `ForNode`, `WhileNode` y `BinaryOpNode`.
- Analisis semantico: `parser.SemanticAnalyzer` valida tipos y reglas basicas.
- Codegen: `codegen.CodeGenerator` recorre el AST con Visitor y genera ASM x86_64.

El backend usa una evaluacion basada en pila, almacena variables en offsets relativos a `rbp` y emite etiquetas para control de flujo.



5 PRUEBAS DE CASOS DE USO

5.1 Caso de uso: compilacion y ejecucion valida

Ejemplo (archivo tests/valid/test_if_elif.py):

```
x = 10
if x > 5:
    print("x es mayor que 5")
    y = 20
elif x > 3:
    print("x es mayor que 3")
else:
    print("x es menor o igual a 3")

z = 15
if z == 15:
    print("z es 15")
```

Compilacion (Windows):

```
java -cp "path/to/antlr4.jar" parser.Main tests/valid/test_if_elif.py > build/ejemplo.asm
```

5.2 Caso de uso: deteccion de errores sintacticos

Ejemplo de error:

```
if x > 5
    print(x)
```

ANTLR reporta errores de sintaxis del tipo mismatched input ... expecting ... cuando faltan tokens obligatorios como : o cuando la estructura no coincide con la gramatica.

5.3 Caso de uso: deteccion de errores semanticos

Ejemplo de variable no definida:

```
print(y)
```

Durante el analisis semantico se reporta un error del tipo [SEMANTIC] Variable no definida: y. Tambien se detectan incompatibilidades de tipos cuando se usan operandos no previstos.

5.4 Caso de uso: inspeccion de salida ensamblador

La salida es ASM x86_64 con seccion de datos y texto. Un fragmento tipico es:

```
section .data
str12345: db "x es mayor que 5", 0x0A, 0

section .text
global _start

_start:
    mov rbp, rsp
    ; ... codigo del if/elif/else ...
```



```

_start_end:
    mov rax, 60
    mov rdi, 0
    syscall

```

El archivo completo se escribe en build/ejemplo.asm cuando se redirige la salida estandar.

6 CONCLUSIONES

6.1 Conclusiones generales del proyecto

El proyecto python2asm cumple exitosamente su objetivo de traducir un subconjunto de Python a ensamblador x86_64 con una arquitectura didactica y clara, implementando el flujo completo de un compilador moderno. informe.md:161-163

6.2 Logros principales

Objetivo cumplido

El compilador traduce efectivamente un subconjunto de Python a ASM x86_64 ejecutable en Linux, demostrando los conceptos fundamentales de construccion de compiladores. informe.md:5-11

Arquitectura modular

- Preprocesamiento de indentacion con PythonIndentPreprocessor.
- Lexer/Parser con ANTLR 4.13.2.
- AST con patron Visitor.
- Generador de codigo x86_64. architecture.md:5-9

Caracteristicas implementadas

- Variables y expresiones aritmeticas (+, -, *, /, %, **).
- Operadores logicos (and, or, not) y comparaciones.
- Estructuras de control: if/elif/else, while, for con range().
- Manejo de indentacion Python con tokens INDENT/DEDENT.
- Funcion print() basica. README.md:45-51

6.3 Estado del proyecto

Componente	Estado
Lexer/Parser	Completo (ANTLR 4.13.2)
AST	Completo (Patron Visitor)

CodeGen	Completo (x86_64 basico)
Testing	Funcional
If/Elif/Else	Completo
While Loops	Completo
For Loops	Completo
Logical Ops	Completo. README.md:117-128

6.4 Limitaciones actuales

Features pendientes

- Funciones definidas por usuario.
- Arrays/Lists y estructuras de datos complejas.
- Argumentos completos para range() (start, stop, step). README.md:129-131

Limitaciones tecnicas

- Analisis semantico basico sin tabla de simbolos completa.
- Manejo de errores no unificado.
- Sin verificacion completa de tipos.
- Evaluacion de operadores logicos sin short-circuit. informe.md:67-74

6.5 Mejoras futuras recomendadas

- Analisis semantico robusto: implementar tabla de simbolos completa y verificacion de tipos.
- Manejo de errores: sistema unificado de reporte de errores.
- Expansion del lenguaje: funciones de usuario, listas, tipos adicionales.
- Optimizaciones: eliminacion de codigo muerto, plegado de constantes y short-circuit evaluation. informe.md:161-163

6.6 Conclusion final

El proyecto es una implementacion educativa exitosa que demuestra los principios fundamentales de compiladores, con un estado funcional y documentado que sirve como base solida para extensiones futuras. README.md:143-144